Subject: ASD&D **Batch:** T11
Semester: V **Name:** Arya Bailur
Class: TE **Roll No:** 8

Experiment 5

Aim: To automate the deployment of an application using Github/Jenkins.

Lab Outcome: Mapped LO3

LO3: Construct and manage automated CI/CD pipelines for building, testing, packaging, and deploying applications.

Theory:

Continuous Integration (CI) and Continuous Deployment (CD) are key DevOps methodologies that bridge the gap between software development and deployment operations.

- **Continuous Integration (CI):** CI is a software development practice where developers regularly merge their code modifications into a central repository. Each merge triggers an automated build and test cycle. This approach enables early detection of bugs, enhances code quality, and minimizes the need for extensive manual testing.
- **Continuous Deployment (CD):** CD refers to the automated workflow of accepting validated code changes, packaging them for release, and pushing them to a production or staging environment without any manual intervention in the deployment process.
- **Jenkins & Declarative Pipelines:** Jenkins is a widely-used open-source automation server that facilitates the automation of building, testing, and deploying software. A Declarative Pipeline employs a structured Groovy-based DSL to specify the pipeline's stages, steps, and runtime environment. These definitions are maintained in a Jenkinsfile, ensuring the pipeline is version-controlled, reproducible, and straightforward to manage.

Step 1: Setting Up and Accessing the Jenkins Server

- Jenkins runs as a background service and needs a compatible Java Development Kit (JDK 21) installed on the system.
- The Jenkins web dashboard is accessible via http://localhost:8080.
- On first launch, an admin account is set up. This account controls Jenkins permissions, job configurations, and overall server management.

Step 2: Verifying the Baseline Pipeline

- A new Jenkins project called **GitHub-Jenkins-Demo** is set up using the Pipeline job type.
- Prior to linking Jenkins with GitHub, a Declarative Pipeline is written directly in the Jenkins job configuration to confirm the execution environment is functional.
- The pipeline employs the **agent any** directive and includes three stages: **Build, Test, and Deploy**. Initially, each stage simply executes an echo command.
- Executing **Build #1** causes Jenkins to run these stages sequentially on an available agent and shows the output in the console. A successful run concludes with a **SUCCESS** status.

Step 3: GitHub Repository and Pipeline-as-Code

- A GitHub repository called **exp5_asd** is created to host the application source code along with the Jenkins automation setup.
- An index.html file is added to act as the frontend of the application.
- A Jenkinsfile is placed at the root of the repository, written in Declarative Pipeline syntax.
- The Deploy stage includes commands that verify or create the deployment directory and transfer index.html from the Jenkins workspace to the target deployment path.

Step 4: Integrating Jenkins with GitHub

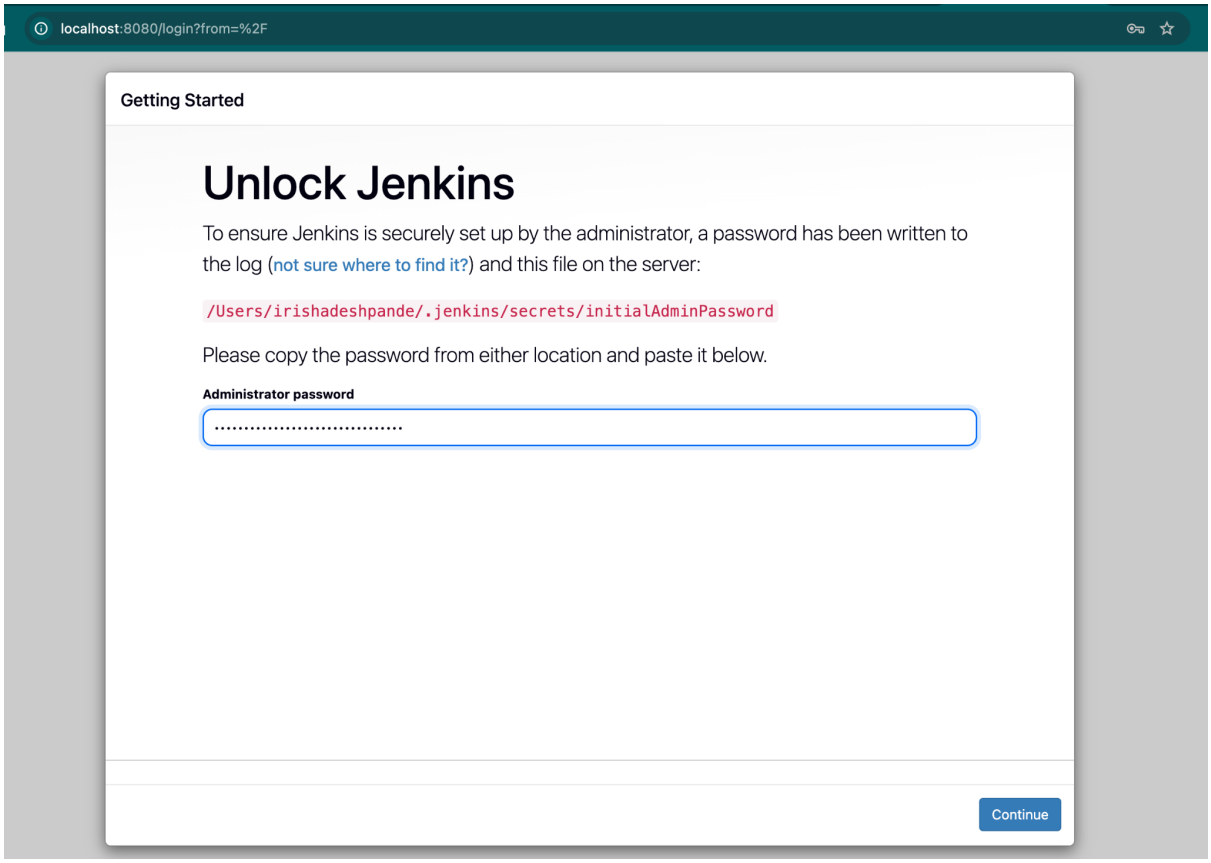
- The Jenkins project configuration is switched from an inline Pipeline script to **Pipeline script from SCM**.
- Git is chosen as the source-code management tool.
- The HTTPS URL of the GitHub repository is provided, the branch is set to */main, and the script path points to the Jenkinsfile.
- Upon running **Build #2**, Jenkins fetches the most recent code from GitHub, checks out the designated branch, reads the Jenkinsfile, and runs the pipeline stages.
- The deployed index.html is then opened in a browser to verify that the initial application has been deployed correctly.

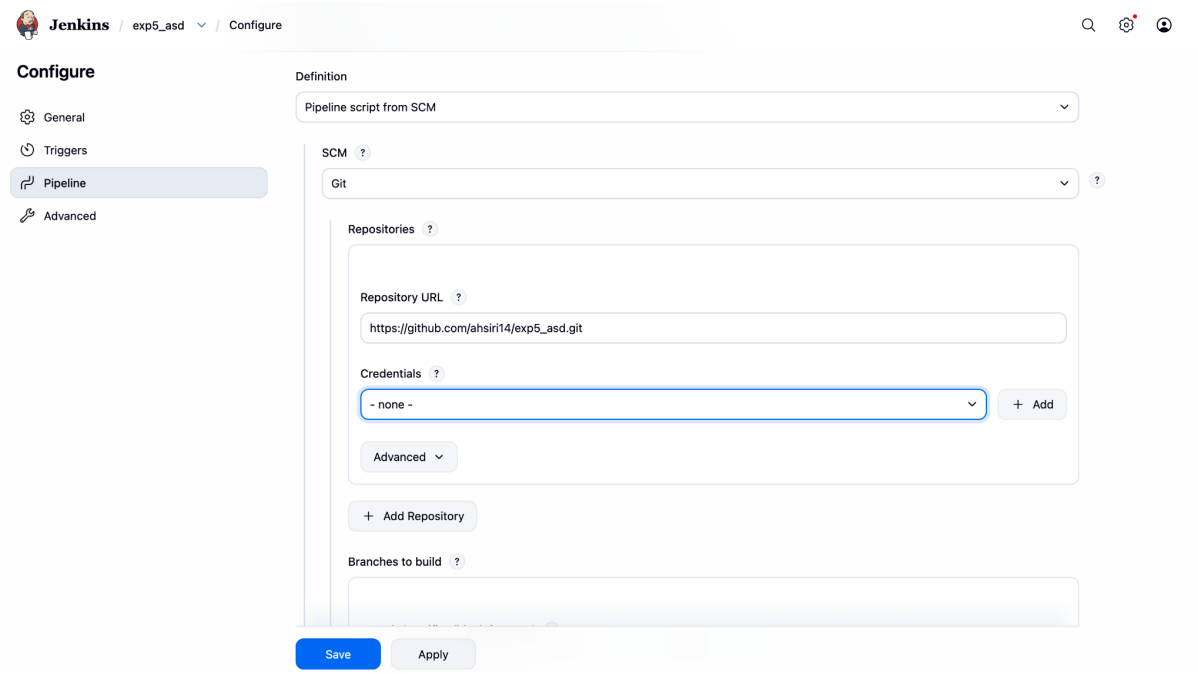
Step 5: Validating the CI/CD Workflow

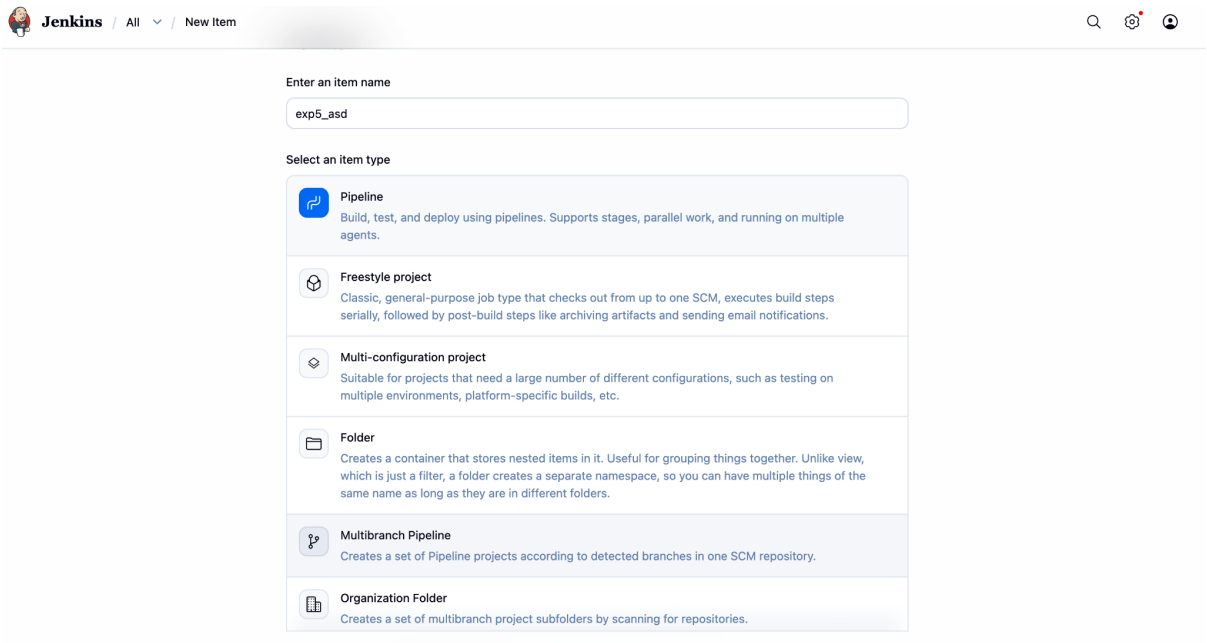
- To showcase continuous deployment, index.html is edited directly on GitHub. CSS styles are applied and the content is updated to reflect that **Version 2.0** has been released via the CI/CD pipeline.
- When **Build #3** runs, Jenkins pulls the modified code from GitHub and re-executes the Build, Test, and Deploy stages.

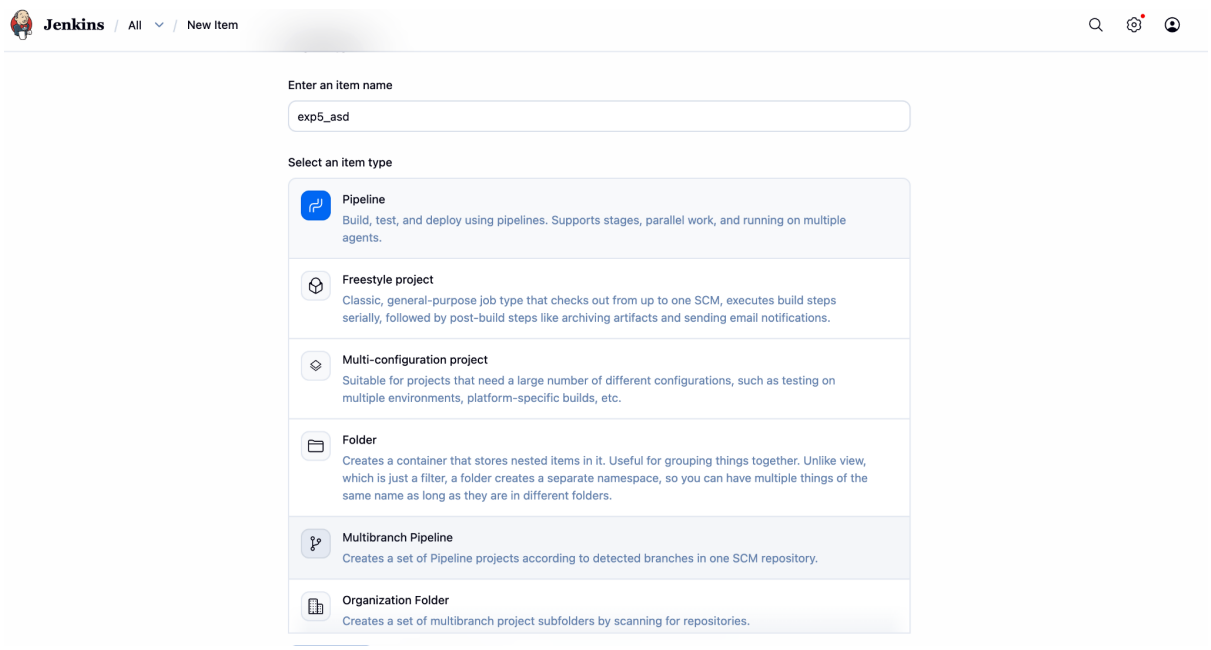
- The updated index.html overwrites the earlier version in the deployment directory.
- Refreshing the browser shows the new design and the **Version 2.0** label, confirming the end-to-end flow from a code update in GitHub through Jenkins automation to live deployment.

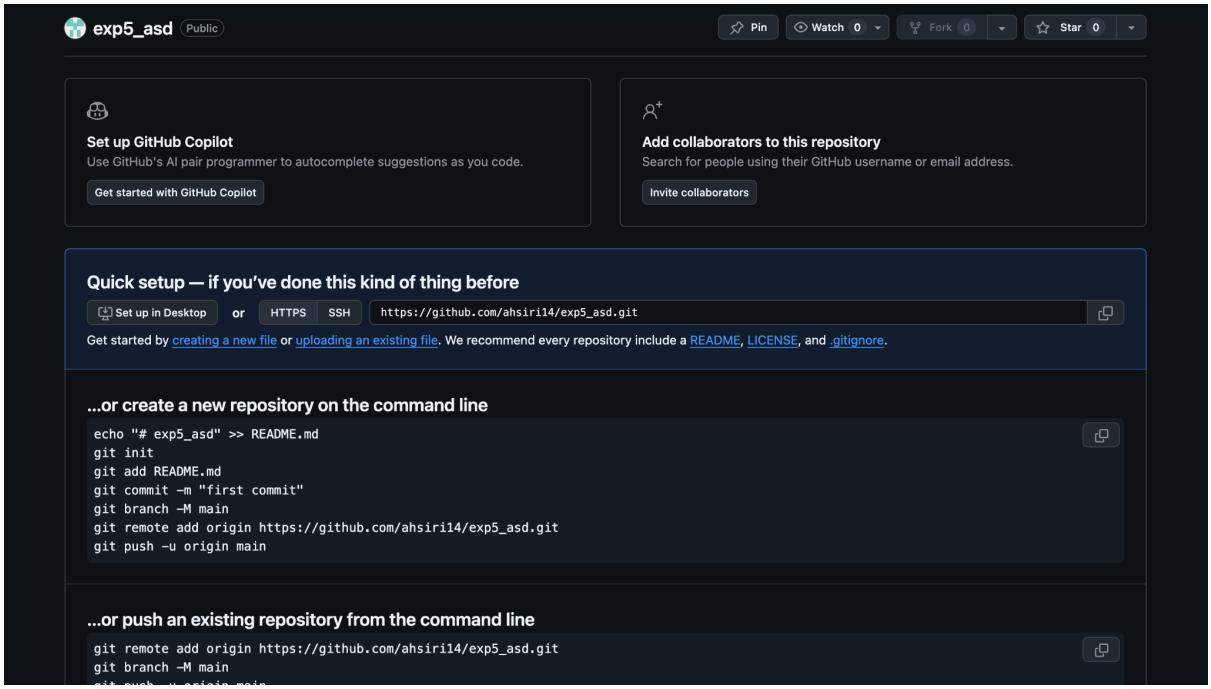
Execution:

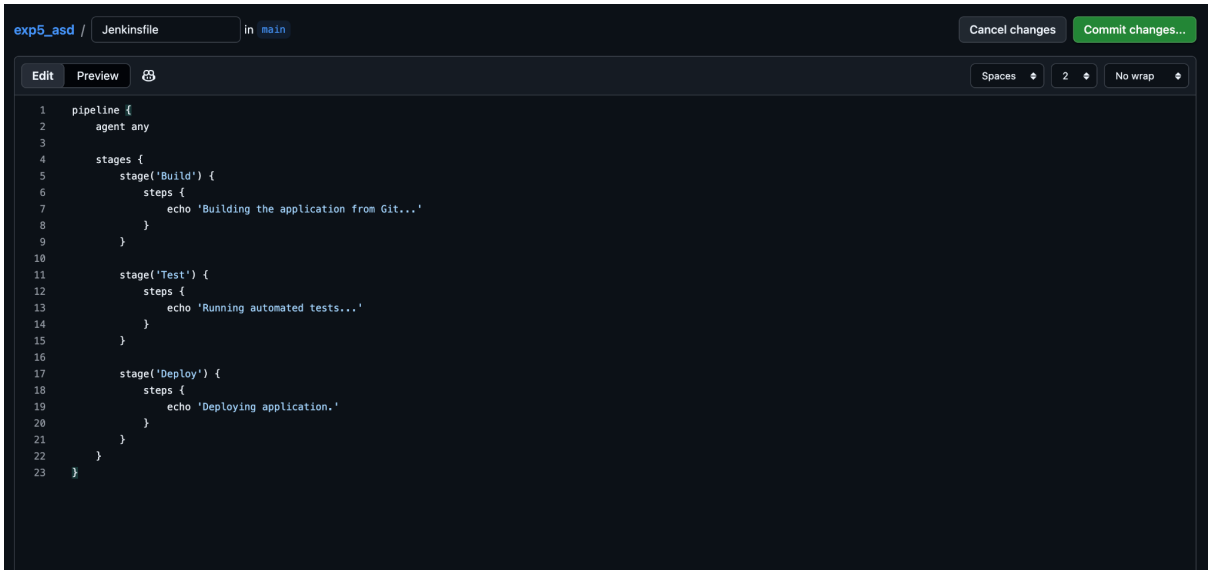



Jenkins / exp5_asd / Configure
Configure
General
Triggers
Pipeline
Advanced
Definition
Pipeline script from SCM
SCM
Git
Repositories
Repository URL
https://github.com/ahsiri14/exp5_asd.git
Credentials
- none -
Add
Advanced
Add Repository
Branches to build
Save
Apply


Jenkins / All / New Item
Enter an item name
exp5_asd
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.
Organization Folder
Creates a set of multibranch project subfolders by scanning for repositories.


Jenkins / All / New Item
Enter an item name
exp5_asd
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.
Organization Folder
Creates a set of multibranch project subfolders by scanning for repositories.


exp5_asd
Public
Pin
Watch 0
Fork 0
Star 0
Set up GitHub Copilot
Use GitHub's AI pair programmer to autocomplete suggestions as you code.
Get started with GitHub Copilot
Add collaborators to this repository
Search for people using their GitHub username or email address.
Invite collaborators
Quick setup — if you've done this kind of thing before
Set up in Desktop
or
HTTPS
SSH
https://github.com/ahsiri14/exp5_asd.git
Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.
...or create a new repository on the command line
echo "# exp5_asd" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/ahsiri14/exp5_asd.git
git push -u origin main
...or push an existing repository from the command line
git remote add origin https://github.com/ahsiri14/exp5_asd.git
git branch -M main



- Status
- Changes
- Console Output
- Delete build '#1'
- Pipeline Overview
- Edit build information
- Replay
- Pipeline Steps
- Workspaces
- Next Build

Console

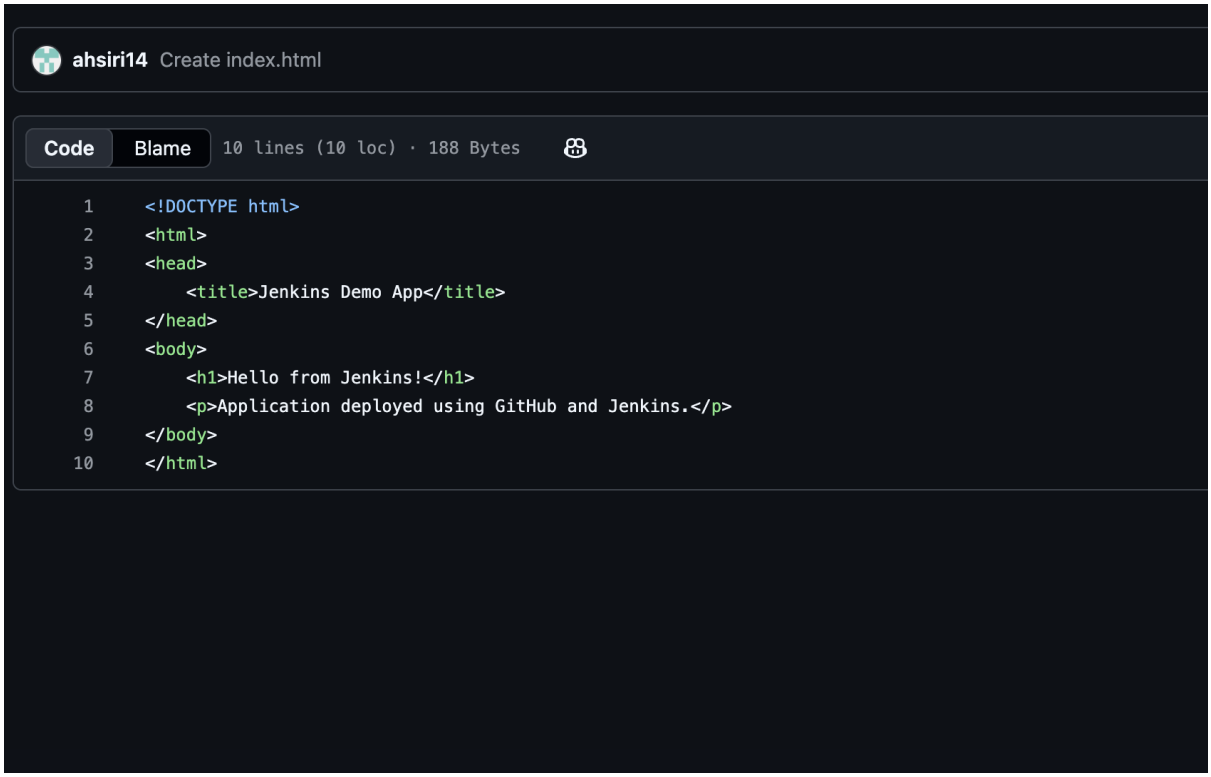
Download | Copy | View as plain text

```
Started by user Irisha Deshpande
hudson.plugins.git.GitException: Command "git fetch --tags --force --progress --prune -- origin
+refs/heads/master:refs/remotes/origin/master" returned status code 128:
stdout:
stderr: fatal: couldn't find remote ref refs/heads/master

        at PluginClassLoader for git-
client//org.jenkinsci.plugins.gitclient.CliGitAPIImpl.launchCommandIn(CliGitAPIImpl.java:2852)
        at PluginClassLoader for git-
client//org.jenkinsci.plugins.gitclient.CliGitAPIImpl.launchCommandWithCredentials(CliGitAPIImpl.java:2193)
        at PluginClassLoader for git-
client//org.jenkinsci.plugins.gitclient.CliGitAPIImpl$1.execute(CliGitAPIImpl.java:638)
        at PluginClassLoader for git//jenkins.plugins.git.GitSCMFileSystem$BuilderImpl.build(GitSCMFileSystem.java:404)
Caused: java.io.IOException
        at PluginClassLoader for git//jenkins.plugins.git.GitSCMFileSystem$BuilderImpl.build(GitSCMFileSystem.java:409)
        at PluginClassLoader for scm-api//jenkins.scm.api.SCMFileSystem.of(SCMFileSystem.java:219)
        at PluginClassLoader for workflow-
cps//org.jenkinsci.plugins.workflow.cps.CpsScmFlowDefinition.create(CpsScmFlowDefinition.java:123)
        at PluginClassLoader for workflow-
cps//org.jenkinsci.plugins.workflow.cps.CpsScmFlowDefinition.create(CpsScmFlowDefinition.java:70)
        at PluginClassLoader for workflow-job//org.jenkinsci.plugins.workflow.job.WorkflowRun.run(WorkflowRun.java:342)
        at hudson.model.ResourceController.execute(ResourceController.java:97)
        at hudson.model.Executor.run(Executor.java:456)
Finished: FAILURE
```

Jenkins 2.568.2

```
 > git config core.sparsecheckout # timeout=10
 > git checkout -f e1115a521dcf0055f4b1f802ecba2f61da8c068f # timeout=10
Commit message: "Create Jenkinsfile"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building the application from Git...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Running automated tests...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying application.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
```





Conclusion:

This experiment effectively illustrated how Jenkins and GitHub can be combined to set up a fundamental CI/CD pipeline. A Declarative Pipeline was defined using a Jenkinsfile comprising Build, Test, and Deploy stages. Jenkins was linked to the GitHub repository and successfully pulled the application source code. The deployment workflow was automated such that modifications to index.html could be built, tested, and deployed through Jenkins. The updated Version 2.0 application was subsequently verified, showcasing how CI/CD enables the automated delivery of code changes from a version control system to the deployment environment.